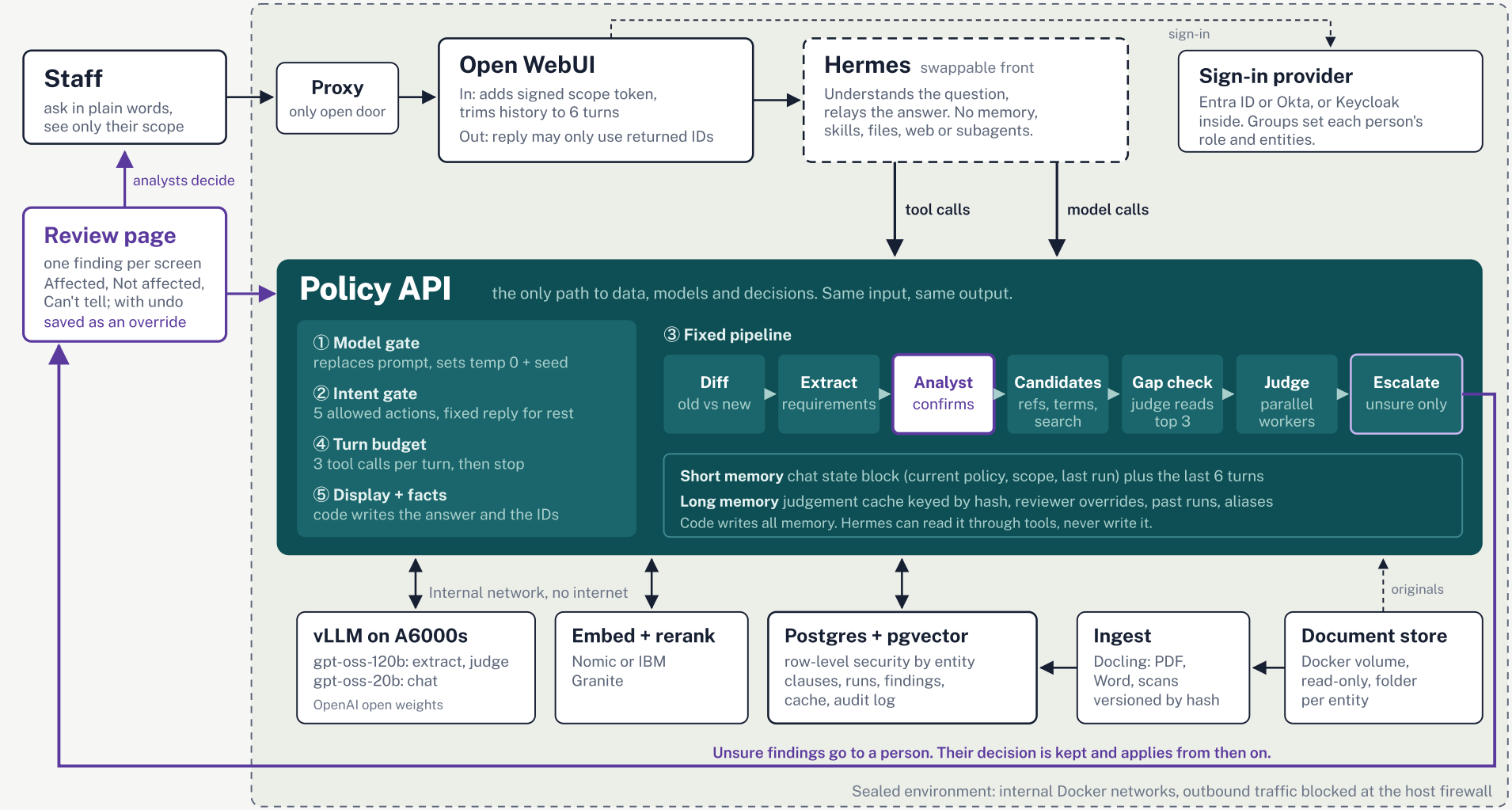
Policy Impact System

When an authority changes a policy, this shows which policies of the entities beneath it are affected. Hermes talks to people; the Policy API decides everything else.

Architecture, draft v0.3
20 October 2026
US-made open models, fully self-hosted



control layer · a person decides · can be replaced without changing anything else · sealed environment

How it stays on task

The design choices behind the diagram, the rules that keep answers repeatable, and the order to build it in.

Policy Impact System
Architecture, draft v0.3

The Policy API keeps Hermes in line

① **Model gate.** Hermes's model calls go through the Policy API, not straight to vLLM. It replaces whatever system prompt Hermes builds with the approved one, adds the signed identity to each tool call where the model can't see it, sets temperature 0 and a fixed seed, removes tools that aren't on the list, and logs every call.

② **Intent gate.** Five allowed actions: `find_policy`, `analyze_impact`, `query_run`, `list_runs`, `explain_finding`. Policy IDs, entities and aliases are matched in code. Anything else gets a fixed reply.

③ **Fixed pipeline.** The same steps every time, with judgements cached by hash. A repeated question gets the same answer.

④ **Turn budget.** At most 3 tool calls per chat turn, enforced by the Policy API whatever Hermes's own settings say.

⑤ **Display and facts.** Code writes the answer and returns the IDs it used. Open WebUI replaces any reply that cites an ID not in the result.

Hermes is the swappable front

Hermes only talks to the Policy API, so it can be swapped for an Open WebUI Pipe in about a day if an upgrade, a security issue or the model gate causes trouble. Its memory, skills, subagents, file and web tools and self-compression are all switched off; its settings files are read-only and checked by hash at start-up.

Changed from the whiteboard

- Hermes no longer reads documents directly, so access rules and the audit log always apply.
- Hermes no longer calls Judge, Extract or Escalate; the Policy API runs them in order.
- Subagents are parallel judge workers the Policy API starts.

What happens when someone asks

1. **Diff** the old and new authority policy, clause by clause.
2. **Extract requirements**: what must happen, by whom, by when.
3. **An analyst confirms** the list before anything runs. A missed requirement would otherwise go unchecked everywhere.
4. **Find candidates** in each entity's policies: clauses that cite the authority policy, use its defined terms, or match by search.
5. **Gap check**: if the judge finds none of the top 3 candidates covers the requirement, it's a gap.
6. **Judge** runs in parallel workers. Clauses with identical text share one judgement; similar wording never does.
7. **Escalate** unsure findings, to the larger model first and then to the review page, sorted by risk. Admins are alerted when the queue grows or items wait too long.

Each entity gets one result per requirement

Conflict	Their policy now says the opposite
Gap	Nothing in their policies covers it
Stale reference	They cite a section that changed or was removed
Compliant	They already meet it or go further
Unsure	Goes to a person on the review page

Memory

Short. Old answers become one-line stubs; a small chat-state block and the last 6 turns are sent. Under 8,000 tokens a turn.

Long. Judgement cache, reviewer overrides, past runs, approved aliases. In Postgres, limited by access, never written by the model.

Who can see and do what

Admin	Manages users, roles and entities
Authority analyst	Runs analyses across all entities beneath their authority, reviews findings
Entity manager	Uploads and views their own entity's documents and findings
Viewer	Reads their own entity's findings

Roles and entities come from groups in the sign-in provider. Open WebUI signs a short-lived token for each message. The Policy API checks it, and Postgres row-level security filters every query, so the model can never widen anyone's access.

Models and GPUs

2 × A6000	gpt-oss-120b across both cards for everything
3 × A6000	120b on two cards for extract and judge; 20b chat and embeddings on the third
4 × A6000	As 3, with the fourth card for ingest, rerank and a spare chat copy

All open-weight models are US-made: OpenAI gpt-oss, Nomic or IBM Granite embeddings, IBM Docling. Put the 120b model on an NVLink-bridged pair.

Build order

1. vLLM running gpt-oss on the A6000s, with tool calls and JSON output tested.
2. Postgres, ingest and a sample of real documents.
3. The Policy API pipeline, scored on 15 past policy changes with known impacts.
4. Hermes, locked down, behind the model gate.
5. Open WebUI, sign-in, filters and the review page.
6. Containment, access and off-topic tests.

Not yet verified: gpt-oss on Ampere in the pinned vLLM version; Hermes with the model gate